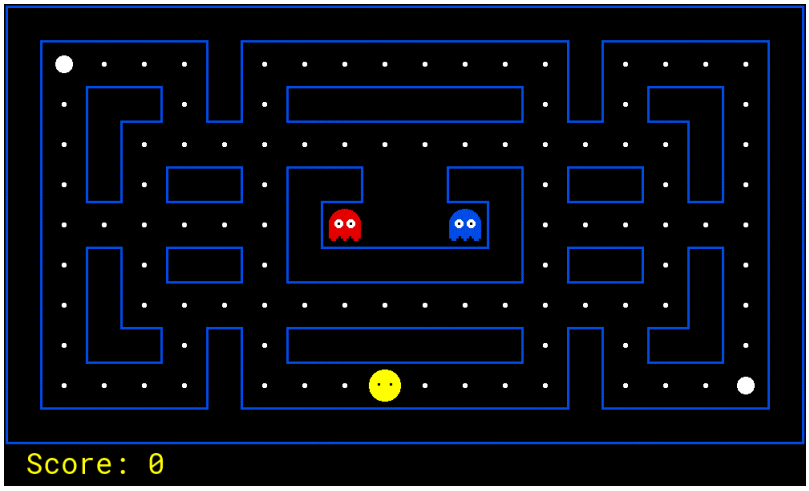
Project 2: Multi-Agent Pac-Man



Pac-Man, now with ghosts.
Minimax, Expectimax,
Evaluation.

Introduction

In this project, you will design agents for the classic version of Pac-Man, including ghosts. Along the way, you will implement both minimax and expectimax search and try your hand at evaluation function design.

The code for this project consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some you can glance over.

Submission

You will fill in portions of `pacai.student.multiagents` during this assignment. You should **only** submit this file.

This assignment should be submitted with the filename **`solution.zip`** HERE. Please use the command line `zip` tool to make sure your submission gets zipped correctly:

```
zip -j solution.zip pacai/student/multiagents.py
```

For these submissions, unzip should directly yield the source files and **not** a directory named "solution", "pN", or "pacai". Note that this is counter to standard conventions when sending a zip file to a human, but easier for the autograder.

After submitting, the autograder will grade your submission and report the result back to you. Do not close the tab or you will not be able to see your score. You can submit as many times as you want. However, if we find you continually making tiny changes instead of testing locally, then we will deduct points. Any attempt to trick the autograder is considered cheating.

Evaluation

Your code will be autograded for technical correctness. Please *do not* change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder (and points will be deducted!). However, you are allowed to add any new classes or function that you need. The correctness of your implementation -- not the autograder's output -- will be the final judge of your score. If necessary, we will review and grade assignments individually to ensure that you receive due credit for your work. This assignment is graded out of 25 points. 20 points will be for correctness as determined by the autograder and the point system given below for each problem. 5 points will be for style, which the autograder will also check. You can run the style checker using the `run_style.sh` script in the project root.

Academic Dishonesty

We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. These cheat detectors are quite hard to fool, so please don't try. We trust you all to submit your own work only; *please* don't let us down. If you do, we will pursue the strongest consequences available to us.

Getting Help

You are not alone! If you find yourself stuck on something, contact the course staff for help. Office hours, section, and Piazza are there for your support; please use them. If you can't make our office hours, let us know and we will schedule more. We want these projects to be rewarding and instructional, not frustrating and demoralizing. But, we don't know when or how to help unless you ask. One more piece of advice: if you don't know what a variable does or what kind of values it takes, print it out.

Running Code on Your Local Machine

In order to run the Pac-Man code on your local machine, you must have Tk, python >= 3.5, and pip installed. Finally you want to install Pacai's package requirements listed in the `requirements.txt` file in the project's root directory:

```
pip3 install --user -r requirements.txt
```

For the next set of instructions, simply follow the steps listed below depending on your OS.

Linux

Install the Python binding for the Tk package, usually called something like `python3-tk`. On Ubuntu you can use the following command:

```
sudo apt-get install python3-tk
```

Mac OS X

The required additional components (Tk and Python3 Tk bindings) are typically bundled with the Python3 package.

Windows

There are two separate methods for getting Pacai to work on Windows. One is using the Windows Subsystem for Linux (WSL) and the other is using Git Bash. We will first talk about the WSL.

Windows Subsystem for Linux (WSL)

1. First, make sure you have the WSL installed on your local machine. You can follow this installation guide. You are allowed to pick any distribution, however we will be providing specific instructions for Ubuntu. Other distributions will have similar steps.

2. Next, you want to make sure you have an X server running. This allows the WSL to launch graphical applications. We recommend using VcXsrv and following this installation guide. Make sure your X server is running whenever you want to run Pacai with graphics.

3. Now launch your WSL. You can do this by typing "Windows Subsystem for Linux" in your start menu and clicking on the icon.

4. Next, you must configure your bash inside the WSL to use the X server from step #2. To do this, use the following commands:

```
echo "export DISPLAY=localhost:0.0" >> ~/.bashrc
source ~/.bashrc
```

This sets your `DISPLAY` environmental variable in your bashrc and then reloads your bashrc.

5. Let's make sure you have all the required packages:

```
sudo apt-get update
sudo apt-get install python3 python3-pip python3-tk x11-apps
```

7. In order to test that you have the graphics set up correctly, you can use the `xeyes` command:

```
xeyes
```

8. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

Git Bash

1. Install Git Bash, you can follow this installation guide.

2. Now launch your Git Bash. You can do this by typing "Git Bash" in your start menu and clicking on the icon.

3. If you are using a newer version of Windows, there may be a conflict with the version of Python 3 installed from the Windows Store. This conflict will cause a permission denied error when running Python 3 in Git Bash. To avoid this error, you will want to disable the Windows Store version of Python. To do this, type "manage app execution aliases" into your start menu and click on the icon. Within the app execution aliases, disable both `python.exe` and `python3.exe`.

4. You may also want to create an alias for `python3`. All the commands in the instructions use `python3`, but Git Bash just refers to the executable as `python`. To create the alias, you can use the following commands:

```
echo "alias python3=python" >> ~/.bash_profile
source ~/.bash_profile
```

5. Finally, clone the Pacai repository and install Pacai's package requirements:

```
pip3 install --user -r requirements.txt
```

Code

All the code for this (and later projects) is available in this repository: https://github.com/linqs/pacman. The only files you should edit are located in the pacai.student package. You should **not** use any third-party libraries, but the Python Standard Library is fair-game. If a bug is found in the code (non-student) code, then the class will be alerted and you will have to pull the changes from this repository.

Any commands provided throughout these instructions are to be executed from the project root directory (the one with the `README.md` and `LICENSE.md` files).

There are many files that will be used throughout this quarter-long project. Below are a few that you should focus on for this assignment.

- Agents
  - pacai.agents.base.BaseAgent
  - pacai.student.multiagents.ReflexAgent
- Actions / Directions
  - pacai.core.directions.Directions
  - pacai.core.actions.Actions
- States
  - pacai.core.gamestate.AbstractGamestate
  - pacai.bin.pacman.PacmanGameState
  - pacai.core.agentstate.AgentState
- Search
  - pacai.core.search.problem.SearchProblem
  - pacai.core.search.position.PositionSearchProblem
  - pacai.core.search.food.FoodSearchProblem
- Data Structures
  - pacai.util.stack
  - pacai.util.queue
  - pacai.util.prioirtyQueue

Multi-Agent Pac-Man

First, play a game of classic Pac-Man:

```
python3 -m pacai.bin.pacman
```

Now, run using a new agent found in pacai.student.multiagents.ReflexAgent by:

```
python3 -m pacai.bin.pacman --pacman ReflexAgent
```

Note that it plays quite poorly even on simple layouts:

```
python3 -m pacai.bin.pacman --pacman ReflexAgent --layout testClassic
```

Inspect its code and make sure you understand what it's doing.

Question 1 (3 points)

Improve the ReflexAgent to play respectably. Reflex agent provides some helpful methods that call into GameState for information. A capable reflex agent will have to consider both food locations and ghost locations to perform well. Your agent should easily and reliably clear the `testClassic` layout:

```
python3 -m pacai.bin.pacman --pacman ReflexAgent --layout testClassic
```

Try out your reflex agent on the default `mediumClassic` layout with one and two ghosts:

```
python3 -m pacai.bin.pacman --pacman ReflexAgent --num-ghosts 1
```

```
python3 -m pacai.bin.pacman --pacman ReflexAgent --num-ghosts 2
```

How does your agent fare? It will likely often die with 2 ghosts on the default board, unless your evaluation function is quite good.

Notes:

- You can never have more than two ghosts on `mediumClassic`.
- As features, try the reciprocal of important values (such as distance to food) rather than just the values themselves.
- The evaluation function you're writing is evaluating state-action pairs; in later parts of the project, you'll be evaluating states.

Options: Default ghosts are random; you can also play for fun with slightly smarter directional ghosts using `--ghosts DirectionalGhost`. If the randomness is preventing you from telling whether your agent is improving, you can use `--seed [NUMBER]` to run with a seed. You can also play multiple games in a row with `--num-games [NUMBER]`. Turn off graphics with `--null-graphics` to run lots of games quickly.

The autograder will check that your agent can rapidly clear the `openClassic` layout ten times without dying more than twice or thrashing around infinitely (i.e. repeatedly moving back and forth between two positions, making no progress).

```
python3 -m pacai.bin.pacman --pacman ReflexAgent --layout openClassic --num-games 10 --
null-graphics
```

Don't spend too much time on this question, though, as the meat of the project lies ahead.

Question 2 (5 points)

Now you will write an adversarial search agent in pacai.student.multiagents.MinimaxAgent. Your minimax agent should work with any number of ghosts. This means you will have to write an algorithm that is slightly more general than what appears in the textbook. In particular, your minimax tree will have multiple min layers (one for each ghost) for every max layer.

Your code should also expand the game tree to an arbitrary depth. Score the leaves of your minimax tree with the evaluation function supplied by the parent class: MultiAgentSearchAgent.getEvaluationFunction(). You can invoke it like: `self.getEvaluationFunction()(state)`.

Important: A single search **ply** is considered to be one Pac-Man move and all the ghosts' responses, so depth 2 search will involve Pac-Man and each ghost moving two times.

Hints and Observations:

- The evaluation function in this part is already written. You shouldn't change this function, but recognize that now we're evaluating **states** rather than actions, as we were for the reflex agent. Look-ahead agents evaluate future states whereas reflex agents evaluate actions from the current state.

- The minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, and -492 for depths 1, 2, 3, and 4 respectively. Note that your minimax agent will often win (665/1000 games for us) despite the dire prediction of depth 4 minimax.

  ```
  python3 -m pacai.bin.pacman --pacman MinimaxAgent --layout minimaxClassic --agent-args depth=4
  ```

- To increase the search depth achievable by your agent, remove the Directions.STOP action from Pac-Man's list of possible actions. Depth 2 should be pretty quick, but depth 3 or 4 will be slow. Don't worry, the next question will speed up the search somewhat.

- Pac-Man is always agent 0, and the agents move in order of increasing agent index.

- All states in minimax should be a PacmanGameState. They are either passed in to MinimaxAgent.getAction or generated via PacmanGameState.generateSuccessor. In this project, you will not be abstracting to simplified states.

- On larger boards such as `openClassic` and `mediumClassic`, you'll find Pac-Man to be good at not dying, but quite bad at winning. He'll often thrash around without making progress. He might even thrash around right next to a dot without eating it because he doesn't know where he'd go after eating that dot. Don't worry if you see this behavior, question 5 will clean up all of these issues.

- When Pac-Man believes that death is unavoidable, they will try to end the game as soon as possible because of the constant penalty for living. Sometimes, this is the wrong thing to do with random ghosts, but minimax agents always assume the worst:

  ```
  python3 -m pacai.bin.pacman --pacman MinimaxAgent --layout trappedClassic --agent-args depth=3
  ```

  Make sure you understand why Pac-Man rushes the closest ghost in this case.

Question 3 (4 points)

Make a new agent that uses alpha-beta pruning to more efficiently explore the minimax tree in pacai.student.multiagents.AlphaBetaAgent. Again, your algorithm will be slightly more general than the pseudo-code in the textbook. Part of the challenge is to extend the alpha-beta pruning logic appropriately to multiple minimizer agents.

You should see a speed-up (perhaps depth 3 alpha-beta will run as fast as depth 2 minimax). Ideally, depth 3 on `smallClassic` should run in just a few seconds per move or faster.

```
python3 -m pacai.bin.pacman --pacman AlphaBetaAgent --agent-args depth=3 --layout smallClassic
```

The AlphaBetaAgent minimax values should be identical to the MinimaxAgent minimax values. Although the actions it selects can vary because of different tie-breaking behavior. Again, the minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, and -492 for depths 1, 2, 3, and 4 respectively.

Question 4 (4 points)

Random ghosts are of course not optimal minimax agents, and so modeling them with minimax search may not be appropriate. Fill in pacai.student.multiagents.ExpectimaxAgent, where the expectation is according to your agent's model of how the ghosts act. To simplify your code, assume you will only be running against a RandomGhost. These ghosts choose amongst their legal actions uniformly at random.

You should now observe a more cavalier approach in close quarters with ghosts. In particular, if Pac-Man perceives that he could be trapped but might escape to grab a few more pieces of food, he'll at least try. Investigate the results of these two scenarios:

```
python3 -m pacai.bin.pacman --pacman AlphaBetaAgent --layout trappedClassic --agent-args depth=3 --null-graphics --num-games 10
```

```
python3 -m pacai.bin.pacman --pacman ExpectimaxAgent --layout trappedClassic --agent-args
depth=3 --null-graphics --num-games 10
```

You should find that your ExpectimaxAgent wins about half the time, while your AlphaBetaAgent always loses. Make sure you understand why the behavior here differs from the minimax case.

Question 5 (4 points)

Write a better evaluation function for pacman in the provided function pacai.student.multiagents.betterEvaluationFunction. The evaluation function should evaluate states, rather than actions like your reflex agent evaluation function did. You may use any tools at your disposal for evaluation, including your search code from the last project. With depth 2 search, your evaluation function should clear the `smallClassic` layout with two random ghosts more than half the time and still run at a reasonable rate. To get full credit, Pac-Man should be averaging around 1000 points when he's winning.

```
python3 -m pacai.bin.pacman --layout smallClassic --pacman ExpectimaxAgent --agent-args
evalFn=pacai.student.multiagents.betterEvaluationFunction --null-graphics --num-games 10
```

Document your evaluation function! We're very curious about what great ideas you have, so don't be shy. We reserve the right to reward bonus points for clever solutions and show demonstrations in class.

Hints and Observations:

- As for your reflex agent evaluation function, you may want to use the reciprocal of important values (such as distance to food) rather than the values themselves.
- One way you might want to write your evaluation function is to use a linear combination of features. That is, compute values for features about the state that you think are important. Then combine those features by multiplying them by different values and adding the results together. You might decide what to multiply each feature by based on how important you think it is.

Mini Contest (3 points extra credit)

Pac-Man's been doing well so far, but things are about to get a bit more challenging. This time, we'll pit Pac-Man against smarter foes in a trickier maze. In particular, the ghosts will actively chase Pac-Man instead of wandering around randomly and the maze features more twists and dead-ends! Extra pellets are given to give Pac-Man a fighting chance. You're free to have Pac-Man use any search procedure, search depth, and evaluation function you like. The only limit is that games can last a maximum of 3 minutes (with graphics off), so be sure to use your computation wisely. We'll run the contest with the following command:

```
python3 -m pacai.bin.pacman --layout contestClassic --pacman ContestAgent --ghosts
DirectionalGhost --null-graphics --num-games 10
```

The three students with the highest score will receive 3, 2, and 1 extra credit points respectively and can look on with pride as their Pac-Man agents are shown off in class. Details: we run 10 games, games longer than 3 minutes get score 0, lowest and highest 2 scores discarded, the rest averaged. Be sure to document what your agent is doing, as we may award additional extra credit to creative solutions even if they're not in the top 3.

Project 2 is done. Go Pac-Man!